

11-Annotation(注解)

鸣谢：黑马程序员



💡 Tip

Previous Chapter(上一章): 10-Reflection(反射)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

Next Chapter(下一章): 12-Dynamic Proxy(动态代理)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

• 11-Annotation(注解)

一、概述

二、自定义注解

1.格式

2.特殊属性: `value`

3.注解的原理

三、元注解

1. `@Target`
2. `@Retention`

四、注解的解析

1. 步骤
2. 代码演示
 - 2.1 需求
 - 2.2 代码实现

五、注解的应用场景——案例：模拟 JUnit 框架

1. 需求分析
2. 代码实现

一、概述

注解(`Annotation`)是Java代码里的特殊标记，比如`@Override`、`@Test`，它们的作用是让其他程序根据注解信息来决定如何执行该程序。

注解可以用于类、接口、构造器、成员变量、方法、参数等位置处。

二、自定义注解

1. 格式

```
1 public @interface 注解名 {  
2     public 属性类型 属性名() default 默认值;  
3 }
```

代码示例如下：

- `MyAnnotation`：

```

1 public @interface MyAnnotation {
2     String a();
3     boolean b() default true;
4     String[] c();
5 }

```

- **Test1:**

```

1 package annotation;
2
3 @MyAnnotation(a = "zsh", c = {"html", "css", "js"})
4 public class Test1 {
5
6     @MyAnnotation(a = "zjl", b = false, c = {"Java", "Python"})
7     public void test1() {
8
9     }
10
11     public static void main(String[] args) {
12
13     }
14 }

```

2.特殊属性： **value**

如果注解中只有一个 **value** 属性或者其他属性都有默认值，那么使用注解时， **value** 字段名可以省略。

代码示例如下：

- **MyAnnotation2:**

```

1 public @interface MyAnnotation2 {
2     String value(); // 特殊属性
3 }

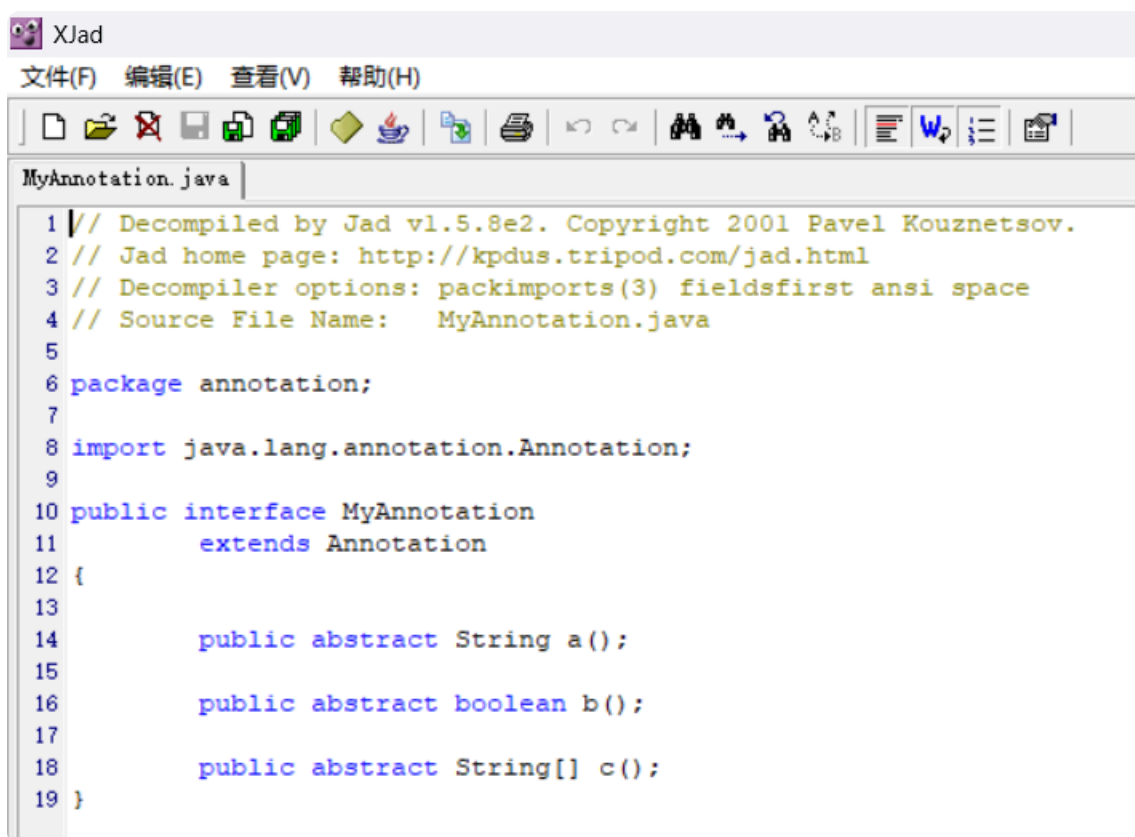
```

- **Test1:**

```
@MyAnnotation(a = "zsh", c = {"html", "css", "js"}) n
// @MyAnnotation2(value = "god")
@MyAnnotation2("god")
public class Test1 {
```

3. 注解的原理

我们执行一下 `Test1` 的 `main` 方法，然后使用 `XJad` 反编译工具对 `MyAnnotation.class` 字节码文件进行反编译并查看内容：



The screenshot shows the XJad application window. The title bar says 'XJad'. The menu bar includes '文件(F)', '编辑(E)', '查看(V)', and '帮助(H)'. The toolbar contains various icons for file operations and editing. The main text area displays the decompiled Java code for 'MyAnnotation.java'. The code is as follows:

```
1 // Decompiled by Jad vl.5.8e2. Copyright 2001 Pavel Kouznetsov.
2 // Jad home page: http://kpdus.tripod.com/jad.html
3 // Decompiler options: packimports(3) fieldsfirst ansi space
4 // Source File Name:  MyAnnotation.java
5
6 package annotation;
7
8 import java.lang.annotation.Annotation;
9
10 public interface MyAnnotation
11     extends Annotation
12 {
13
14     public abstract String a();
15
16     public abstract boolean b();
17
18     public abstract String[] c();
19 }
```

分析反编译后的字节码文件，可知注解本质上是一个继承了 `Annotation` 这个注解接口的接口。

而我们在主程序中使用注解，其实相当于创建了注解的一个实现类对象。

三、元注解

 Important

元注解就是修饰注解的注解。

1. @Target

作用： 声明被修饰的注解只能在哪些位置被使用。

格式： `@Target(ElementType.???, ElementType.???, ...)`

???	说明
TYPE	类和接口
FIELD	成员变量
METHOD	成员方法
PARAMETER	方法参数
CONSTRUCTOR	构造器
LOCAL_VARIABLE	局部变量

2. @Retention

作用： 声明注解的保留周期。

格式： `@Retention(RetentionPolicy.???, RetentionPolicy.???, ...)`

???	说明
<code>SOURCE</code>	只作用在源码阶段，编译为字节码文件后消失。
<code>CLASS</code> （默认值）	编译为字节码文件后继续存在，运行阶段消失。
<code>RUNTIME</code> （开发常用）	一直保留到运行阶段。

四、注解的解析

Important

判断类、接口、构造器、成员变量、方法、参数等位置处是否存在注解，若存在则把注解里的内容解析出来。

1. 步骤

指导思想：要解析谁上面的注解，就应该先拿到谁。

`Class`、`Constructor`、`Field`、`Method` 都实现了 `AnnotatedElement` 接口，它们都拥有解析注解的能力。

序号	<code>AnnotatedElement</code> 接口提供的解析注解的方法	说明
01	<code>Annotation[] getDeclaredAnnotations()</code>	获取当前对象上的注解。
02	<code>T getDeclaredAnnotation(Class<T> annotationClass)</code>	获取指定的注解对象。
03	<code>boolean isAnnotationPresent(Class<Annotation> annotationClass)</code>	判断当前对象上是否存在某个注解。

2. 代码演示

2.1 需求

1. 定义注解 `MyAnnotation3`，要求：
 - 包含属性：`String value()`，`double a() default 100`，`String[] b`。
 - 限制注解只能使用在类和成员方法上。
 - 注解一直保留到运行阶段。
2. 定义一个类 `Demo`，在类中定义一个 `test` 方法，并在该类和其方法上使用 `MyAnnotation3` 注解。
3. 定义测试类 `AnnotationParser`，用于解析 `Demo` 类中的全部注解。

2.2 代码实现

- `MyAnnotation`：

```
1 import java.lang.annotation.ElementType;
2 import java.lang.annotation.Retention;
3 import java.lang.annotation.RetentionPolicy;
4 import java.lang.annotation.Target;
5
6 @Target({ElementType.TYPE, ElementType.METHOD})
7 @Retention(RetentionPolicy.RUNTIME)
8 public @interface MyAnnotation3 {
9     String value();
10    double a() default 100;
11    String[] b();
12 }
```

- `Demo`：

```

1 | @MyAnnotation3(value = "chatgpt", a = 666.66, b = {"beef",
   | "mutton"})
2 | public class Demo {
3 |
4 |     @MyAnnotation3(value = "gemini", a = 9.99, b = {"dog", "cat"})
5 |     public static void test() {
6 |
7 |     }
8 | }

```

- **AnnotationParser:**

```

1 | import java.lang.annotation.Annotation;
2 | import java.lang.reflect.Method;
3 | import java.util.Arrays;
4 |
5 | public class AnnotationParser {
6 |     public static void main(String[] args) throws
   | NoSuchMethodException {
7 |         parseClass();
8 |         System.out.println();
9 |         parseMethod();
10 |    }
11 |
12 |    public static void parseClass() {
13 |        Class demo = Demo.class;
14 |        if (demo.isAnnotationPresent(MyAnnotation3.class)) {
15 |            MyAnnotation3 annotation = (MyAnnotation3)
   | demo.getDeclaredAnnotation(MyAnnotation3.class);
16 |            System.out.println("value=" + annotation.value());
17 |            System.out.println("a=" + annotation.a());
18 |            System.out.println("b=" +
   | Arrays.toString(annotation.b()));
19 |        }
20 |    }
21 |
22 |    public static void parseMethod() throws NoSuchMethodException {
23 |        Method test1 = Demo.class.getDeclaredMethod("test");
24 |        if (test1.isAnnotationPresent(MyAnnotation3.class)) {
25 |            MyAnnotation3 annotation = (MyAnnotation3)
   | test1.getDeclaredAnnotation(MyAnnotation3.class);
26 |            System.out.println("value=" + annotation.value());
27 |            System.out.println("a=" + annotation.a());

```



```
28         System.out.println("b=" +  
29         Arrays.toString(annotation.b()));  
30  
31     }  
32 }
```

- 控制台输出：

```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwa  
value=chatgpt  
a=666.66  
b=[beef, mutton]  
  
value=gemini  
a=9.99  
b=[dog, cat]  
  
Process finished with exit code 0
```

五、注解的应用场景——案例：模拟 JUnit 框架

1. 需求分析

需求：定义几个方法，只要加了 `@MyTest` 注解，就会触发该方法执行。

分析：

1. 自定义注解 `@MyTest`，只能注解方法，一直存活。
2. 定义几个方法，一部分加上 `@MyTest` 注解修饰，另一部分不加。
3. 模拟一个 `junit` 程序，所有加了 `@MyTest` 注解的方法都会被触发执行（使用暴力反射）。

2.代码实现

- `MyTest`:

```
1 package my_junit;
2
3 import java.lang.annotation.ElementType;
4 import java.lang.annotation.Retention;
5 import java.lang.annotation.RetentionPolicy;
6 import java.lang.annotation.Target;
7
8 @Target(ElementType.METHOD)
9 @Retention(RetentionPolicy.RUNTIME)
10 public @interface MyTest {
11 }
```

- `MyJUnit`:

```
1 package my_junit;
2
3 import java.lang.reflect.InvocationTargetException;
4 import java.util.Arrays;
5
6 public class MyJUnit {
7
8     public static void main(String[] args) {
9
10         MyJUnit myJUnit = new MyJUnit();
11         Class c = MyJUnit.class;
12
13         Arrays.stream(c.getDeclaredMethods()).forEach(method -> {
14             if (method.isAnnotationPresent(MyTest.class)) {
15                 try {
16                     method.setAccessible(true);
17                     method.invoke(myJUnit);
18                 } catch (IllegalAccessException e) {
19                     throw new RuntimeException(e);
20                 } catch (InvocationTargetException e) {
21                     throw new RuntimeException(e);
22                 }
23             }
24         });
25     }
```

```
26
27     public void test1() {
28         System.out.println("===test1===");
29     }
30
31     @MyTest
32     public void test2() {
33         System.out.println("===test2===");
34     }
35
36     public void test3() {
37         System.out.println("===test3===");
38     }
39
40     @MyTest
41     public void test4() {
42         System.out.println("===test4===");
43     }
44 }
```

- 控制台输出:

```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\
===test2===
===test4===

Process finished with exit code 0
|
```